Evidence Collection

1 Background / Problem
In audit and compliance engagements, auditors often must collect their own evidence directly from a client's systems. They may receive read-only access to tools such as Workday (or another HRIS/ERP), GitHub, NetSuite, Jira/Linear, etc. Evidence collection is frequently manual: auditors open links, take screenshots, record fields into spreadsheets, and download supporting artifacts.

We are building a *General Browser Agent* that can perform these workflows reliably across many systems, with consistent outputs suitable for audit documentation.

The product interface and authentication system is up for you to spec out, but mostly we will be evaluating based on the quality of the agent itself.

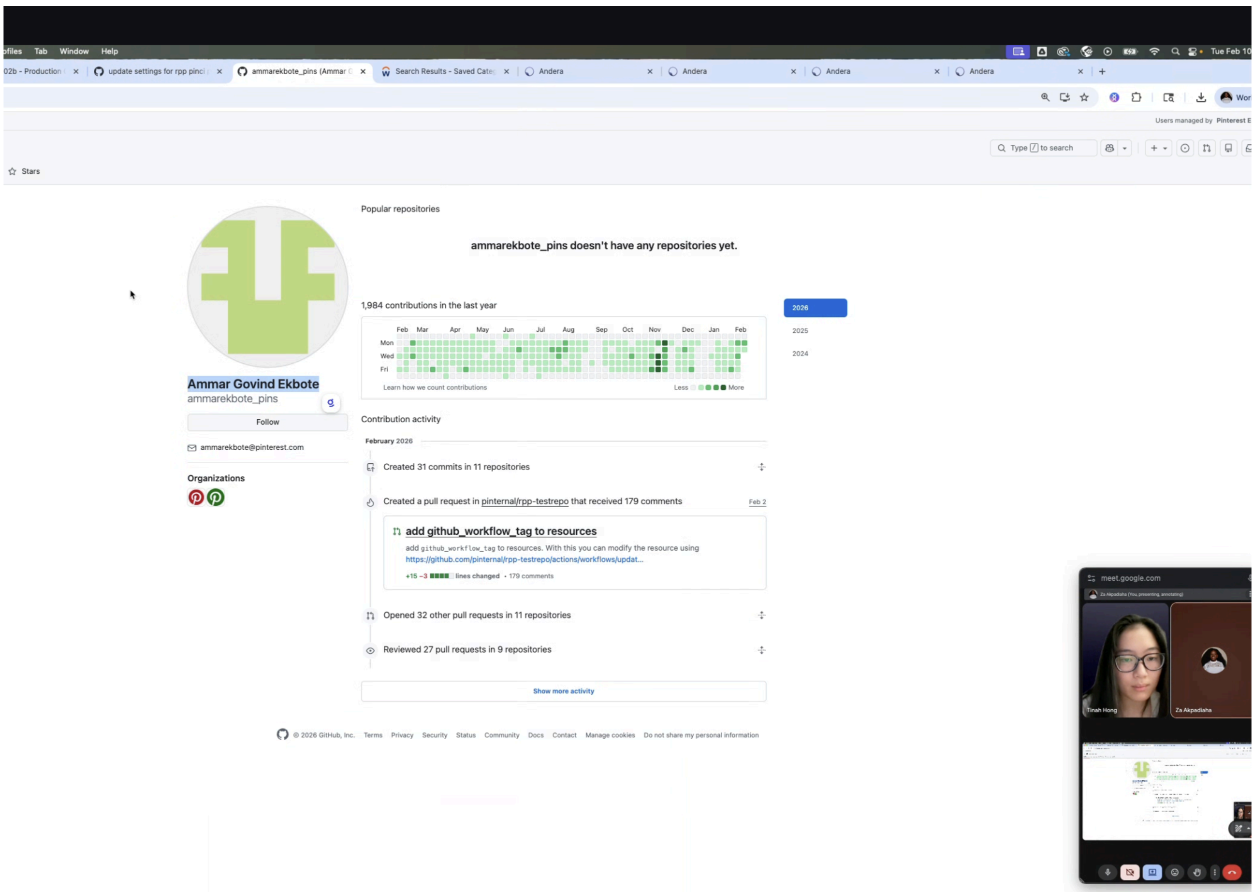
2 Scope and Assumptions
- 2.1 System-Agnostic Requirement
  - We cannot assume Github/Workday specifically and should remain ERP/HRIS-agnostic. The agent must operate using only what is visible in the browser UI, unless an integration is explicitly available for a trial scenario.
- 2.2 Operating Modes
  - The agent must support both: 1) Manual UI mode (default): Navigate in browser, extract values, capture screenshots, download artifacts. 2) Integration-assisted mode (optional): If a system provides an export/listing (e.g., GitHub listing, HR roster export), the agent may use it to accelerate collection, but must still be able to proceed without it.
- 2.3 Evidence Standard
  - Possible output types: screenshots, csvs, judgement, or a mix of all of the above.

3 Deliverables
For every task, the agent must produce:
1. File system evidence with:
   a. One folder per sample (stable `sample_id`)
   b. Screenshots saved with consistent naming
   c. Any downloaded artifacts stored under the sample folder

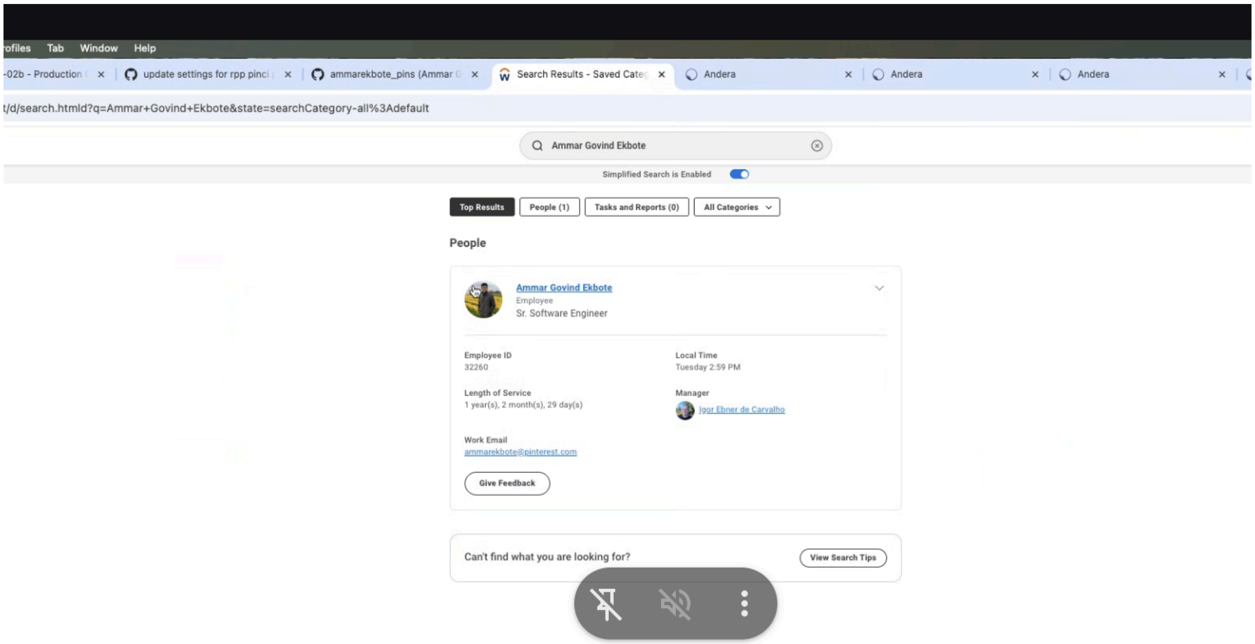
Priorities:
1. Accuracy
2. Generability
3. Scalability to thousands of samples
4. Consistency between samples
5. Speed.

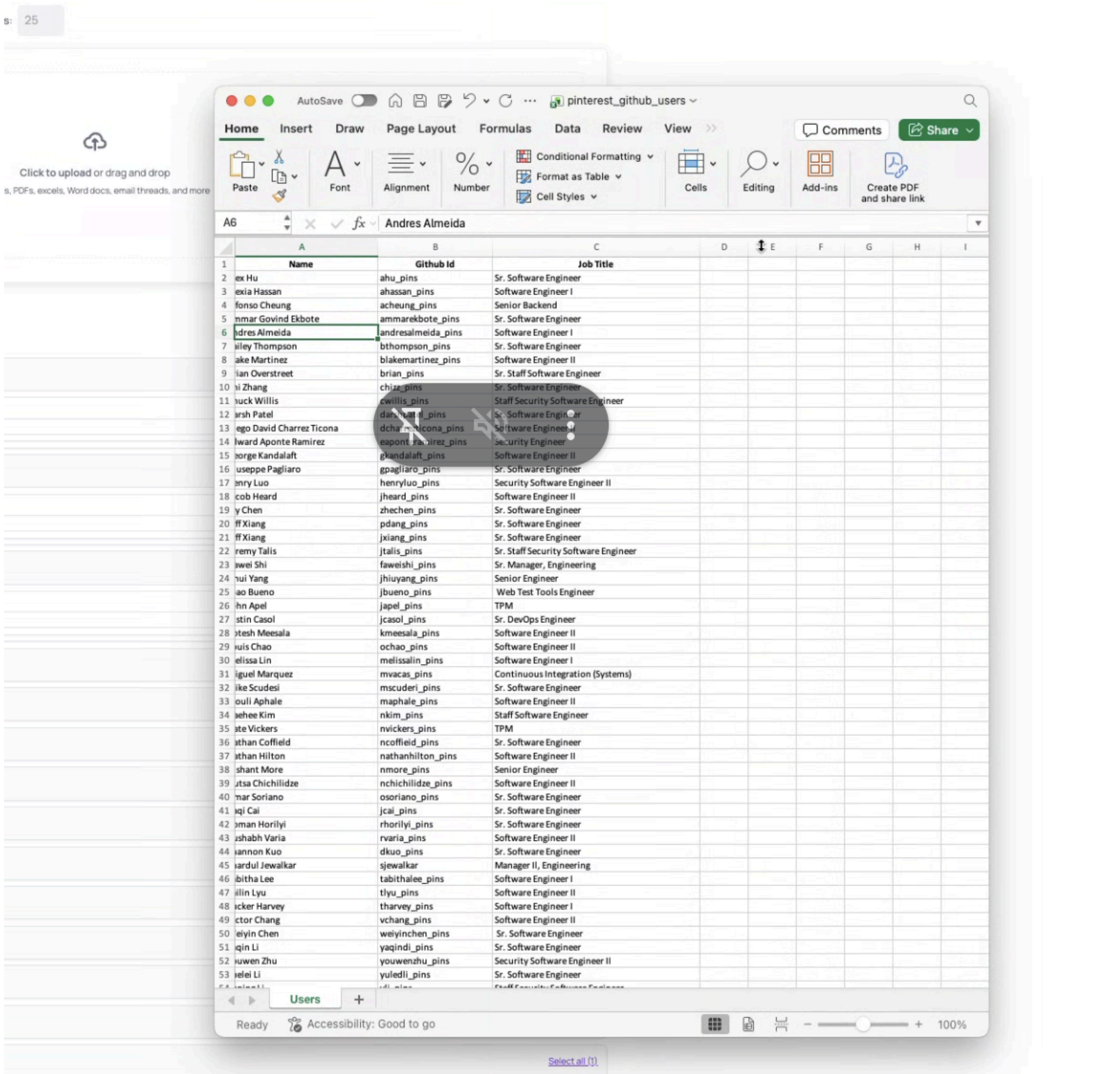
Examples of Tasks:

- Pinterest task for 50-1000 users:
    - Went to their user in github, found their user id, and then their job title from workday
    - Inputted them into a CSV
- Sometimes, this may be easier if you're able to integrate to get a Github or Workday listing, we may assume that we cannot always get these and the agent will have to do it manually, but we should also be able to operate in the case where they do have these



-

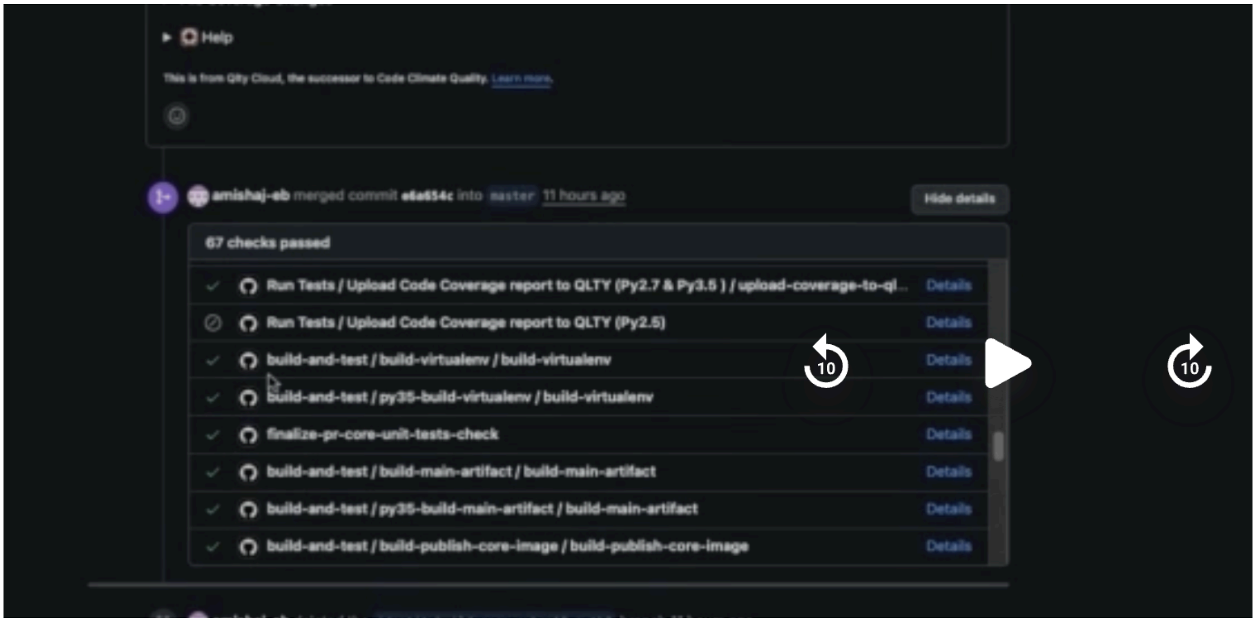

rofiles Tab Window Help
-02b - Production
update settings for rpp pinci
ammarekbote_pins (Ammar
Search Results - Saved Categ
Andera
Andera
Andera
t/d/search.htmld?q=Ammar+Govind+Ekbote&state=searchCategory-all%3Adefault
Ammar Govind Ekbote
Simplified Search is Enabled
Top Results
People (1)
Tasks and Reports (0)
All Categories
People
Ammar Govind Ekbote
Employee
Sr. Software Engineer
Employee ID
32260
Local Time
Tuesday 2:59 PM
Length of Service
1 year(s), 2 month(s), 29 day(s)
Manager
Igor Ebner de Carvalho
Work Email
ammarekbote@pinterest.com
Give Feedback
Can't find what you are looking for?
View Search Tips

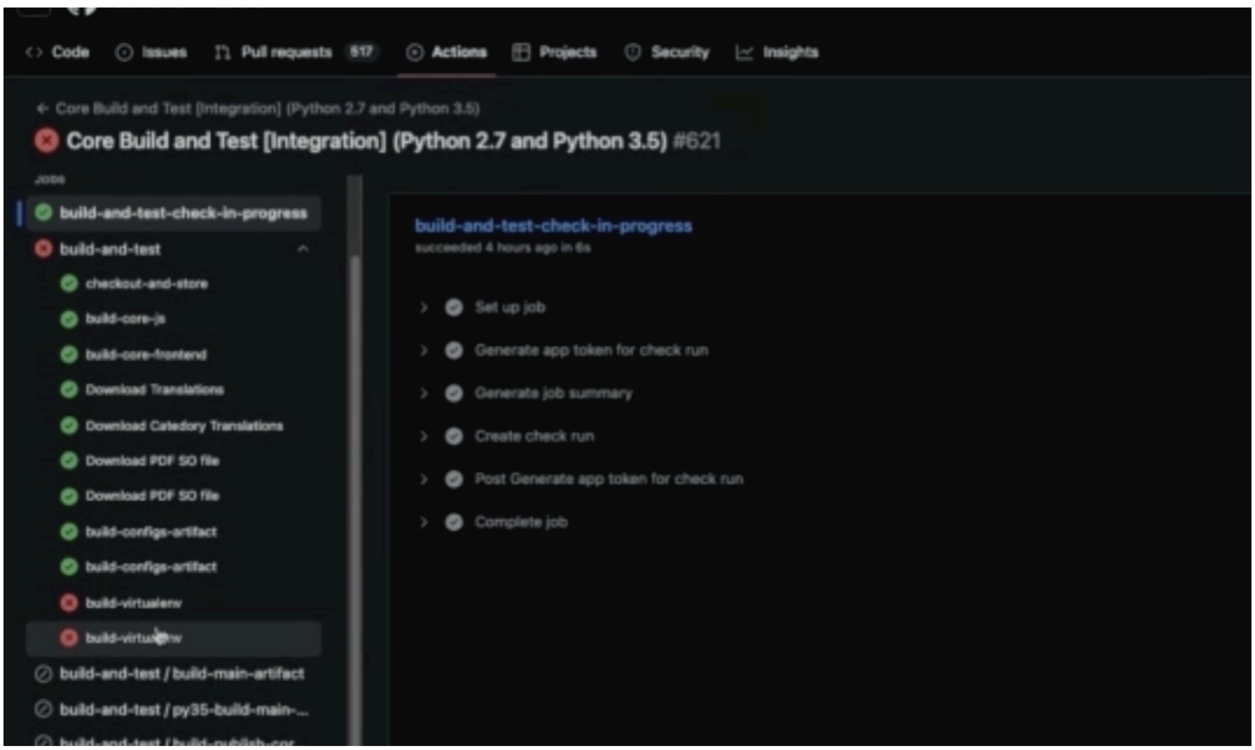


Because we don't have Workday and we want to be agnostic to the ERP, we can come up with a few specific examples that emulate what the General Browser Agent should be able to do.

1. Screenshots
- Input: Excel list of links to linear tickets
- Task: The agent should go through one by one and open the linear ticket URL and take a screenshot. Then it should compile a CSV of the ticket number, who it's assigned to, and the due date.
2. Github Screenshots
    1. Watch the following time snippet for a detailed explanation: 5:07-7:20
    2. [Andera Tool Integration Sync - Circleback]()
3. Task 1. Go through 60 commits, for each of them, take a screenshot of each part of the viewport, open the checks, identify which checks pass or were optional, identify any that were merged with

fails, go into the CI and take that screenshot. If they link to a Jira ticket, go to the Jira ticket and take screenshots of that.

4. 



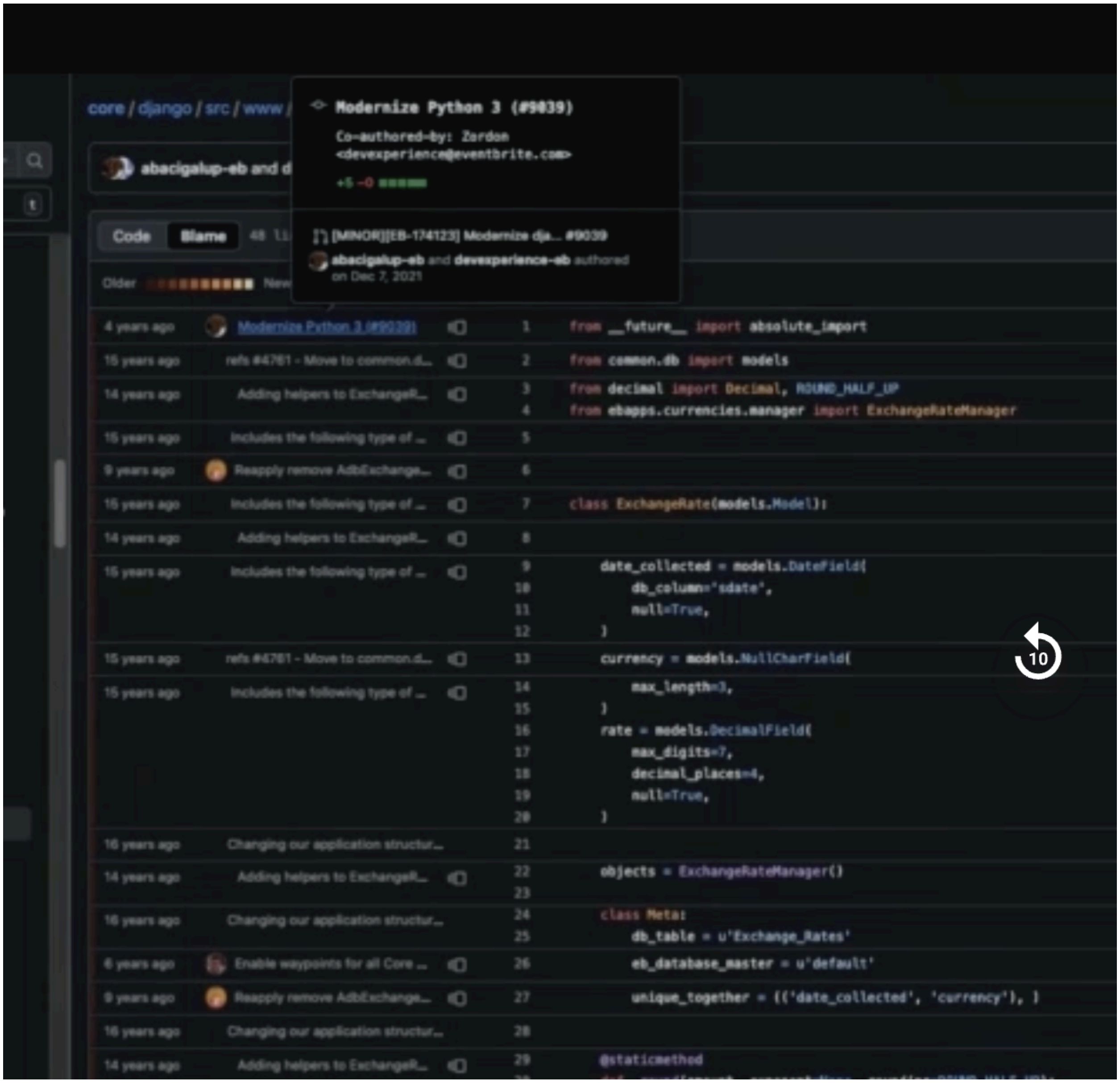
a. Outputs:

1. CSV with github commit, github link, who created PR, who approved, who merged, any notes on failed checks, jira ticket link
2. File directory, folder per sample, screenshots all embedded into the sample folder

3. CSV Combination
    1. You have a CSV of names from an event. Go through each and find their LinkedIn. Add 4 columns to the CSV: LinkedIn URL, School, Where they currently recently work, and how long theyve been there

Task 2: 13:20-14:30, 16:11-17

- Input: code string
- Task: find the file where the code is, switch to Blame View. See how long ago the last recent change to that code was. Determine if within X time (the year). If so, go into the commit and see if it may have materially changed how something was calculated. Flag if so. Take screenshots to document if so. Additionally, if the code wasn't modified in the past year, but there was code below that changes how the function operates that was modified within the last year. Then we have to determine if this could have materially affected the outcome of the function/calculation and document if so



18:01-20:00 fill out form in Workday (take screenshot of it filled out) and download the resulting report, then click on individual table tabs and download all the attachments from there, then (cant do but can think of an analogous example to test)

General thoughts:

- We should be agnostic to ERP, given every company seems to have so many different systems
- We should be able to accomplish complex tasks that have a high number of samples. Thus, we should likely leverage a file system and many subagents